

Python Exception Handling

Top 30 Interview-Level Questions & Answers

Designed for technical interviews: conceptual, scenario-based, output-prediction, debugging, and production-design questions.

How to use: First answer each question yourself, then compare with the model answer. For coding questions, explain both the output and the reason.

Level 1 — Core Interview Fundamentals

Q1. What is the difference between a syntax error and an exception?

Answer: A syntax error occurs when Python cannot parse the program, while an exception occurs during execution. Syntax errors must be fixed before execution; exceptions can often be handled with try-except.

Interview point: Interview point: Example exceptions include ValueError, TypeError and ZeroDivisionError.

Q2. Explain the complete execution flow of try-except-else-finally.

Answer: Python first executes try. If an exception occurs, the remaining try code is skipped and the first matching except handler runs. If no exception occurs, else runs. finally runs regardless of whether an exception occurred, subject to control-flow cases such as process termination.

Interview point: Key order: try → except OR else → finally.

Q3. Why should we catch specific exceptions instead of using a bare except?

Answer: Specific handlers make the program predictable and prevent unrelated programming errors from being hidden. For example, catch ValueError when converting input instead of catching every possible exception.

Interview point: Good practice: except ValueError: ... rather than except: ...

Q4. What is the difference between except Exception and except BaseException?

Answer: Exception is the base class for normal, non-system-exiting exceptions. BaseException is higher in the hierarchy and also includes exceptions such as SystemExit and KeyboardInterrupt. Application code should normally catch Exception or more specific subclasses, not BaseException.

Interview point: Interview point: A broad catch should still be used carefully.

Q5. What happens if an exception occurs inside a try block after some statements have already executed?

Answer: Python stops executing the remaining statements in that try block and searches for a matching except handler. Statements executed before the exception are not automatically rolled back.

Interview point: Example: database/file changes made before the failure may remain unless the application explicitly uses rollback/cleanup logic.

Q6. Can one try block have multiple except blocks? How does Python choose one?

Answer: Yes. Python checks handlers from top to bottom and executes the first compatible handler. Therefore, more specific exception classes should normally appear before their broader base classes.

Interview point: Classic trap: placing `except Exception` before `except ValueError` makes the `ValueError` handler unreachable for `ValueError`.

Q7. What is the difference between `ValueError` and `TypeError`?

Answer: `TypeError` usually means an operation received an inappropriate type. `ValueError` means the type is acceptable but the value is inappropriate.

Interview point: Examples: `10 + '5' → TypeError`; `int('abc') → ValueError`.

Q8. What does the 'as e' part of an except statement do?

Answer: It binds the caught exception object to a variable. That object can be inspected or logged.

Interview point: Example: `except ValueError as e: print(e)`. The exception object also provides information such as args and traceback-related attributes.

Q9. What is exception propagation?

Answer: If a function does not handle an exception, the exception propagates to its caller. This continues outward through the call stack until a compatible handler is found; otherwise the program ends with an unhandled exception.

Interview point: Interview point: A try around a function call can catch an exception raised inside that function.

Q10. What is the purpose of the else block in exception handling?

Answer: `else` runs only when the `try` block completes without an exception. It is useful for code that should execute only after the risky operation succeeds.

Interview point: Why use it? It keeps non-risky success logic outside the protected `try` block, reducing accidental exception catching.

Level 2 — Practical & Tricky Questions

Q11. Why is `finally` useful in real applications?

Answer: `finally` is used for cleanup that should happen whether the protected operation succeeds or fails, such as closing resources or releasing locks.

Interview point: Typical examples: file handles, database resources, locks, and temporary resources.

Q12. Does `finally` always execute?

Answer: In normal Python control flow, `finally` executes whether an exception occurs or not, including when `return`/`break`/`continue` is used. It is not an absolute guarantee against events such as forced process termination or abrupt interpreter shutdown.

Interview point: Interview-safe answer: say 'normally always executes', not 'under every possible circumstance'.

Q13. What happens when return is used in try and another return is used in finally?

Answer: The return in finally overrides the earlier return.

Interview point: Example: try: return 10; finally: return 20 → result is 20.

Note: Warning: This is generally bad practice because it can hide the original return value and even suppress an exception.

Q14. What is raise? Give a practical use case.

Answer: raise explicitly raises an exception. It is useful when input violates a business rule that Python would not automatically consider an error.

Interview point: Example: if age < 0: raise ValueError('Age cannot be negative').

Q15. What is the difference between raise and raise e inside an except block?

Answer: A bare raise re-raises the currently handled exception and is the normal choice when you want to pass it upward. raise e explicitly raises the exception object again and can change the traceback presentation. Prefer bare raise for simple re-raising.

Interview point: Interview point: Re-raise after logging when the caller still needs to handle the failure.

Q16. What is a custom exception and how do you create one?

Answer: A custom exception represents an application-specific failure. It is normally created by subclassing Exception.

Interview point: Example: class InsufficientBalanceError(Exception): pass

Note: Use it when a domain concept deserves its own clear exception type.

Q17. Why should custom exceptions usually inherit from Exception, not BaseException?

Answer: Exception is intended as the base class for normal application exceptions. BaseException also includes system-exiting exceptions that applications generally should not intercept.

Interview point: Recommended pattern: class MyError(Exception): pass.

Q18. What is exception chaining? Explain raise ... from ...

Answer: Exception chaining connects a new exception to the original exception. raise NewError(...) from original tells Python that the new exception was directly caused by the original one.

Interview point: Useful when converting low-level errors into meaningful application-level errors while preserving the cause.

Q19. What is the difference between __context__ and __cause__?

Answer: __context__ records an exception that was being handled when another exception occurred. __cause__ is set explicitly when using raise new_exception from original_exception.

Interview point: Interview point: Explicit chaining with 'from' communicates a direct causal relationship.

Q20. What does 'raise ... from None' do?

Answer: It suppresses the display of the automatically chained context in the traceback while raising the new exception.

Interview point: Use it when the lower-level implementation detail should not be exposed in the displayed exception, while the original context can still exist internally.

Level 3 — Advanced, Scenario & Design Questions

Q21. What is exception handling best practice in production code?

Answer: Catch the narrowest useful exception, provide meaningful handling, log unexpected failures, avoid silently swallowing errors, and clean up resources reliably.

Interview point: Also validate at boundaries and use custom exceptions for important domain failures.

Q22. Why is 'except Exception: pass' considered dangerous?

Answer: It silently ignores failures. The program may continue with invalid state and debugging becomes difficult.

Interview point: Better: catch a specific expected exception and either handle it meaningfully or log and re-raise unexpected failures.

Q23. Can an exception raised inside a called function be caught by the caller's try block?

Answer: Yes. A try block covers exceptions raised during the execution of code it calls, directly or indirectly, unless an inner handler handles the exception first.

Interview point: Example: `try: calculate(); except ZeroDivisionError: ...`

Q24. Predict the output: try/except/else/finally.

Code: `try: print('A'); 10/0; print('B') except ZeroDivisionError: print('C') else: print('D') finally: print('E')`

Interview point: Answer: A, C, E.

Note: Reason: The exception occurs after A, so B is skipped; except runs; else does not run; finally runs.

Q25. Predict the output when except clauses are in the wrong order.

Code: `try: raise ValueError('x') except Exception: print('general') except ValueError: print('value')`

Interview point: Answer: general.

Note: Reason: Exception is a base class that matches ValueError, and the first matching handler wins. The ValueError handler is therefore never reached.

Q26. How would you handle invalid user input without crashing the program?

Answer: Put only the conversion/validation operation that can fail inside try and catch the expected exception.

Interview point: Example: `try: age = int(input('Age: ')); except ValueError: print('Enter a valid integer.')`

Note: Interview point: A loop can repeat input until valid data is received.

Q27. How would you handle a file operation safely?

Answer: Prefer a with statement for files because it manages resource cleanup automatically. Catch expected exceptions such as FileNotFoundError or PermissionError when the application needs to provide a user-friendly response.

Interview point: Example pattern: `try: with open('data.txt') as f: data = f.read(); except FileNotFoundError: ...`

Q28. What is the relationship between IndexError and LookupError?

Answer: IndexError and KeyError are subclasses of LookupError. LookupError is a broader category for invalid key/index lookups.

Interview point: Interview point: Catching LookupError can handle both IndexError and KeyError, but a specific handler is preferable when the two cases need different responses.

Q29. How should exceptions be handled across application layers?

Answer: Handle an exception at the layer that can make a meaningful decision. Lower layers can raise precise exceptions; higher layers can translate them into user-facing messages or API responses.

Interview point: Example: database layer raises a domain/database error; service layer decides recovery; UI/API layer presents a safe message.

Q30. Design question: How would you build robust exception handling for a banking/payment system?

Answer: Define domain exceptions such as InsufficientBalanceError and InvalidPaymentError; validate inputs; catch specific expected failures; use transactions/rollback for database operations; log unexpected exceptions without exposing sensitive data; and re-raise when a higher layer must decide the response.

Interview point: Strong interview answer: 'I would separate expected business failures from unexpected programming/system failures and handle them at the appropriate layer.'

Rapid Revision — Must-Know Interview Points

- **try** → code that may raise an exception
- **except** → handles a matching exception
- **else** → runs only when try succeeds
- **finally** → cleanup code that normally runs regardless of success/failure
- **raise** → explicitly raises an exception
- **raise** inside except → re-raises the current exception
- **Exception** → base class for normal application exceptions
- **BaseException** → higher-level base including SystemExit and KeyboardInterrupt
- **Specific exceptions** are generally preferable to broad catches
- **Custom exceptions** should normally inherit from Exception
- **raise ... from ...** → explicit exception chaining
- **__context__** → implicit exception context; **__cause__** → explicit cause

Interview strategy: When asked an exception-handling question, answer in this order: (1) definition, (2) short example, (3) why it matters in production. For scenario questions, mention specific exceptions, logging, cleanup, and correct responsibility at the application layer.

Technical references: [Python documentation on Errors and Exceptions](#) and [Built-in Exceptions](#).